RICE UNIVERSITY

# GPU Architecture

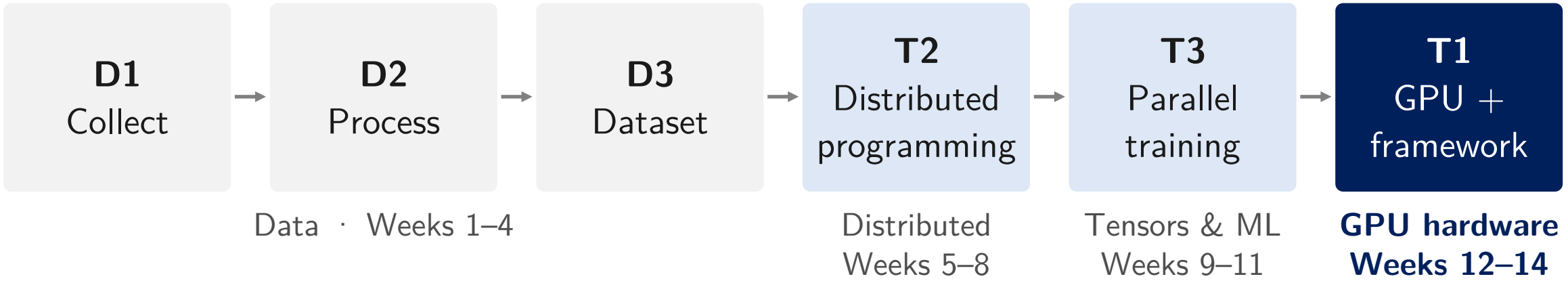
Throughput processors: SMs, warps, blocks, and grids

Yuke Wang

Rice University

COMP 329 · Spring 2027 · Week 12, Lecture 1

# Where we are in the pipeline

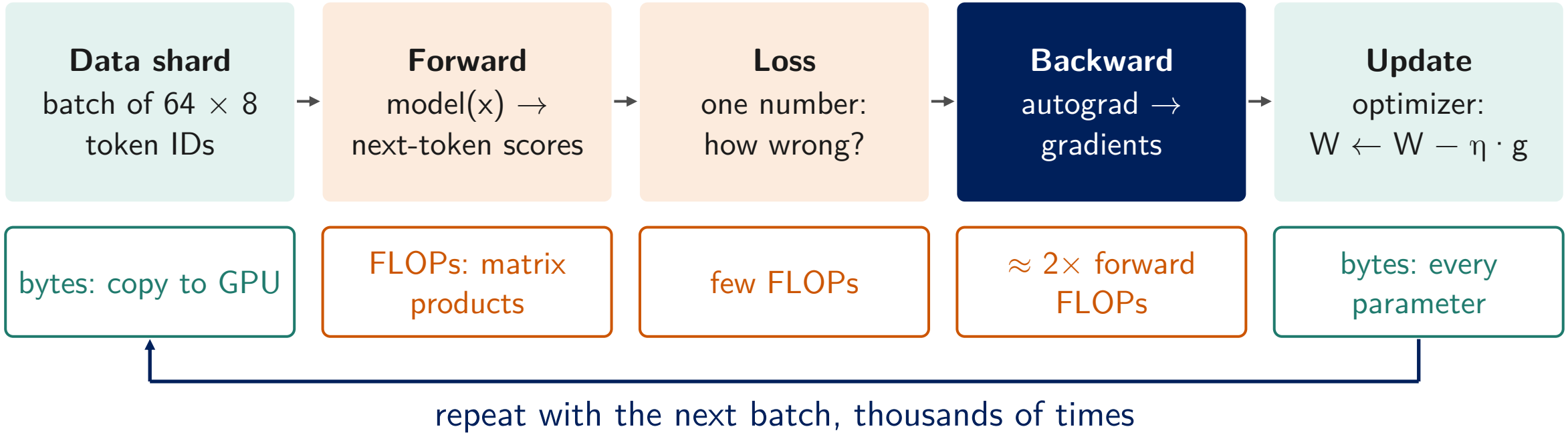



**In our pipeline**

Today: what happens inside **one GPU** running **one kernel** — the foundation for T1 and Lab 3.

Next: one training step again, and where the GPU fits among Weeks 5–14.

# What one training step does




**Key idea**

Training time = number of steps × time per step. Weeks 5–11 split it across **many GPUs**; Weeks 12–14 make **each GPU fast**.

# Weeks 5–14 at a glance

| Problem | | System component | Where |
|---|---|---|---|
| One GPU is too slow or too small | → | **Processes + communication: MPI, NCCL** | Wk 5–8 · T2 |
| How do we split the work? | → | **Data, tensor, pipeline parallelism** | Wk 9–11 · T3 |
| One step is millions of multiply-adds | → | **GPUs and kernels** | Wk 12–14 · T1 |

dark: today

# Why do we need GPUs?

**The job:** train GPT-3 (175 billion parameters)
$\approx$ **3.14 $\times$ $10^{23}$** floating-point operations, almost all multiply-adds

**If one CPU core did all of this work, how long would it take?**

One **CPU core**, at its peak
$\approx$ **52,000 years**

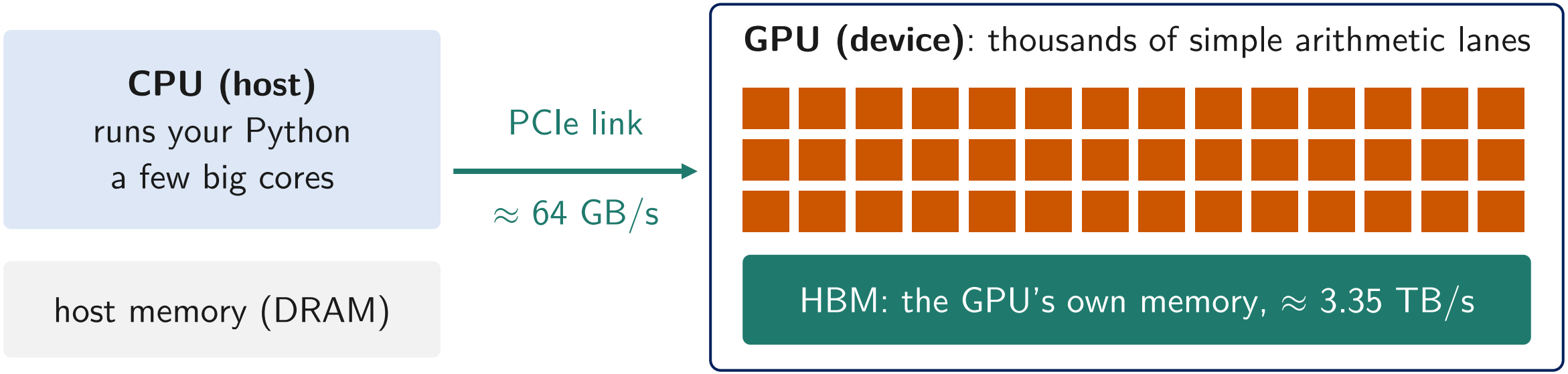
One **H100 GPU**, at its peak
$\approx$ **10 years**

**Key idea**

A GPU is **thousands of times** faster than a CPU core at this work — and even one GPU is too slow. So: many GPUs in T2 and T3, and fast GPUs in T1.

Source: Brown et al., NeurIPS 2020 (GPT-3 compute); CPU: assumed 3 GHz core, 2 AVX-512 FMA units; H100 datasheet (SXM5), BF16 peak.

# What a GPU is, in plain words




1. Copy the data to HBM once, and keep it there.
2. Launch kernels: "run $z = x + y$ on all $10^8$ elements."

**Key idea**

A **kernel**: one function the GPU runs on many elements at once. The CPU **launches** it.

Source: H100 datasheet (SXM5), HBM bandwidth; PCIe 5.0 x16, per direction.

# Where GPUs show up

- **Graphics first:** every pixel is independent, so the chip is many simple cores.
- **2007, CUDA:** those cores become programmable for any arithmetic.
- **2012, AlexNet:** trained on two GPUs. Llama 3 used up to **16,384** H100s.
- **In PyTorch:** `x.to("cuda")`; every op on x then runs as GPU kernels.

**The price**

Fast only with **lots of independent work**

Data must fit in **80 GB** of GPU memory

Each launch costs CPU time: tiny jobs are **overhead-bound**

**In our pipeline**

**Why GPUs?** Training is a huge number of independent multiply-adds, and a GPU runs tens of thousands at once. In T1 and Lab 3 your kernels run on one.

Source: Krizhevsky et al., NeurIPS 2012; Llama Team, "The Llama 3 Herd of Models," 2024; H100 datasheet (SXM5).

# Learning goals for today

- Explain why a GPU trades **latency** for **throughput**, and what Amdahl's law (Week 5) says it cannot fix.
- Name the parts of an **SM**: warp schedulers, registers, **CUDA cores**, **tensor cores**.
- Map **thread** $\rightarrow$ **warp** $\rightarrow$ **block** $\rightarrow$ **grid** onto the SMs, and count blocks, warps, and **waves**.
- Explain **SIMT** and predict the cost of **warp divergence**.

# A puzzle to start

**KERNEL 1**

```
i = thread_index()
if i % 2 == 0:
    y[i] = slow_a(x[i])
else:
    y[i] = slow_b(x[i])
```

**KERNEL 2**

```
i = thread_index()
if (i // 32) % 2 == 0:
    y[i] = slow_a(x[i])
else:
    y[i] = slow_b(x[i])
```

**Same n, half the elements in each branch, both branches equally slow. Which kernel is faster, and by how much?**

**Key idea**

**Kernel 2, about 2× faster.** Threads run in groups of 32: in kernel 1 every group runs **both** branches; in kernel 2 each group runs **one**.

# Today's plan

1

## From CPU to GPU

Why build a processor with thousands of slow, simple cores?

2

## Inside the GPU

What is a GPU made of?

3

## Threads, warps, blocks, and grids

How does one program become 100,000 threads?

**Build**

## Map a grid, cost a branch

Work in pairs, on paper, about eight minutes.

Part 1

# From CPU to GPU

Why build a processor with thousands of slow, simple cores?


**1** · From CPU to GPU

**2** · Inside the GPU

**3** · Threads, warps, blocks, and grids

**Build** · Map a grid, cost a branch

# Latency versus throughput

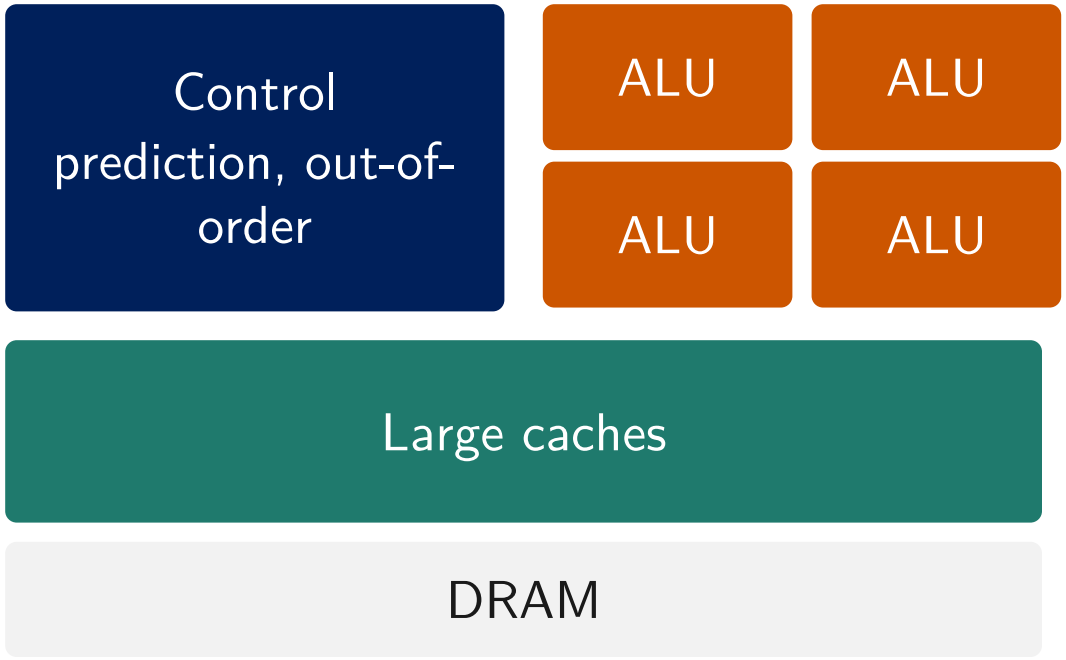
| | **Latency-oriented (CPU)** | **Throughput-oriented (GPU)** |
|---|---|---|
| Goal | Finish **one** task fast | Finish **many** tasks per second |
| Cores | Tens of large cores | Thousands of simple lanes |
| Chip area | Caches, prediction, control | Arithmetic and registers |
| Hides slow memory by | Big caches, speculation | Switching to other threads |

**Key idea**

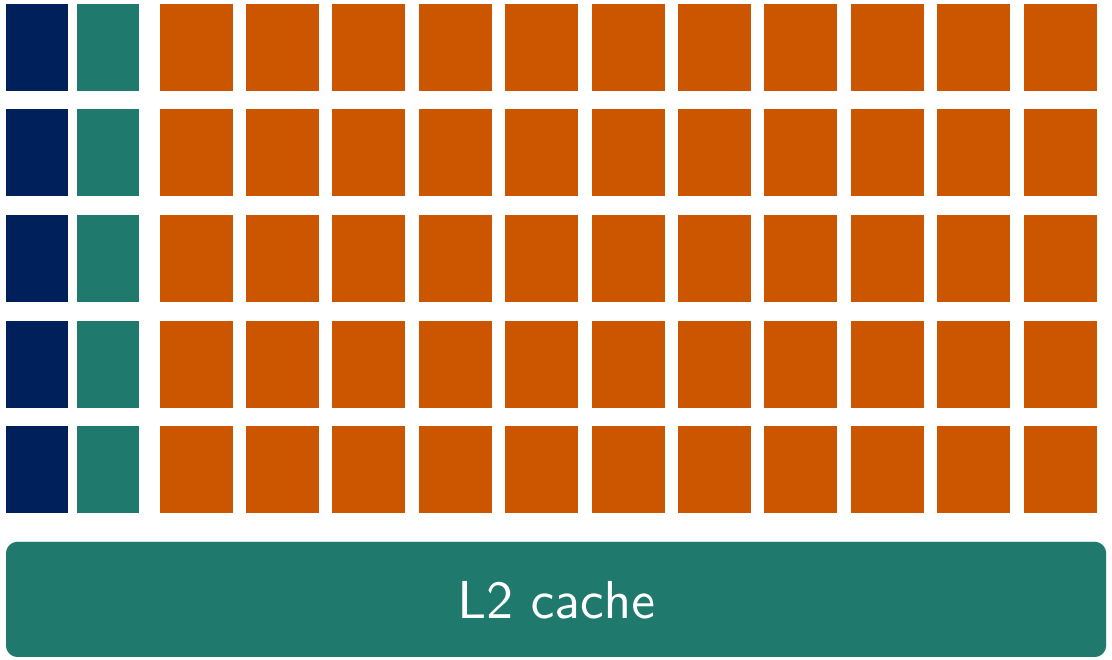
A sports car versus a fleet of buses. Training is a **throughput** problem.

# Where the transistors go

## CPU: a few powerful cores




## GPU: many simple lanes

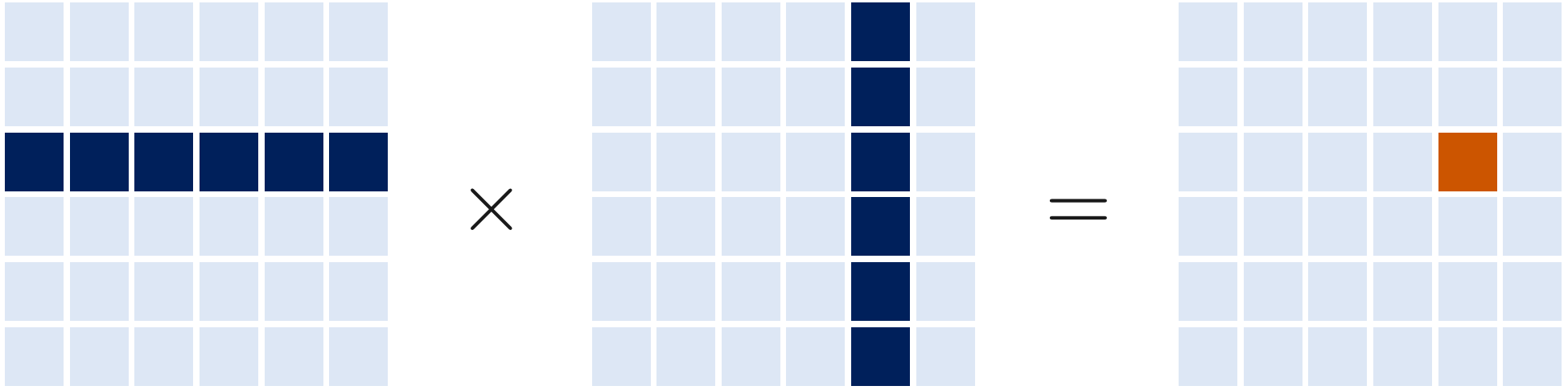



blue: control · green: cache · orange: arithmetic

**Key idea**

The GPU spends its area on arithmetic, so it needs **many threads** in flight to stay busy.

# Why training fits the GPU



**A** row i × **B** column j = **C** C[i, j]

**Key idea**

Every C[i, j] is **independent**: $4096^2 \approx$ **16.8 million** tasks that can run at once.

# Recall Amdahl's law (Week 5), on a GPU

$$\text{speedup} = 1 \;/\; (\; (1 - p) + p \;/\; s \;)$$

$$s \to \infty: \quad \text{speedup} \to 1 \;/\; (1 - p)$$

p = 0.95 on the GPU
5% left on the CPU
**at most 20×**

**Watch out**

The CPU side of a step — Python, data loading, **launching kernels** — bounds what any GPU can do. Lecture 2 measures the launch cost.

# Check yourself

Two minutes with a neighbour.

- Is training a latency problem or a throughput problem? Why?
- Name a computation that is hard to parallelize on a GPU, and say why.
- 80% of a program moves to a GPU that runs that part 100× faster. What is the overall speedup?

Part 2

# Inside the GPU

What is a GPU made of?

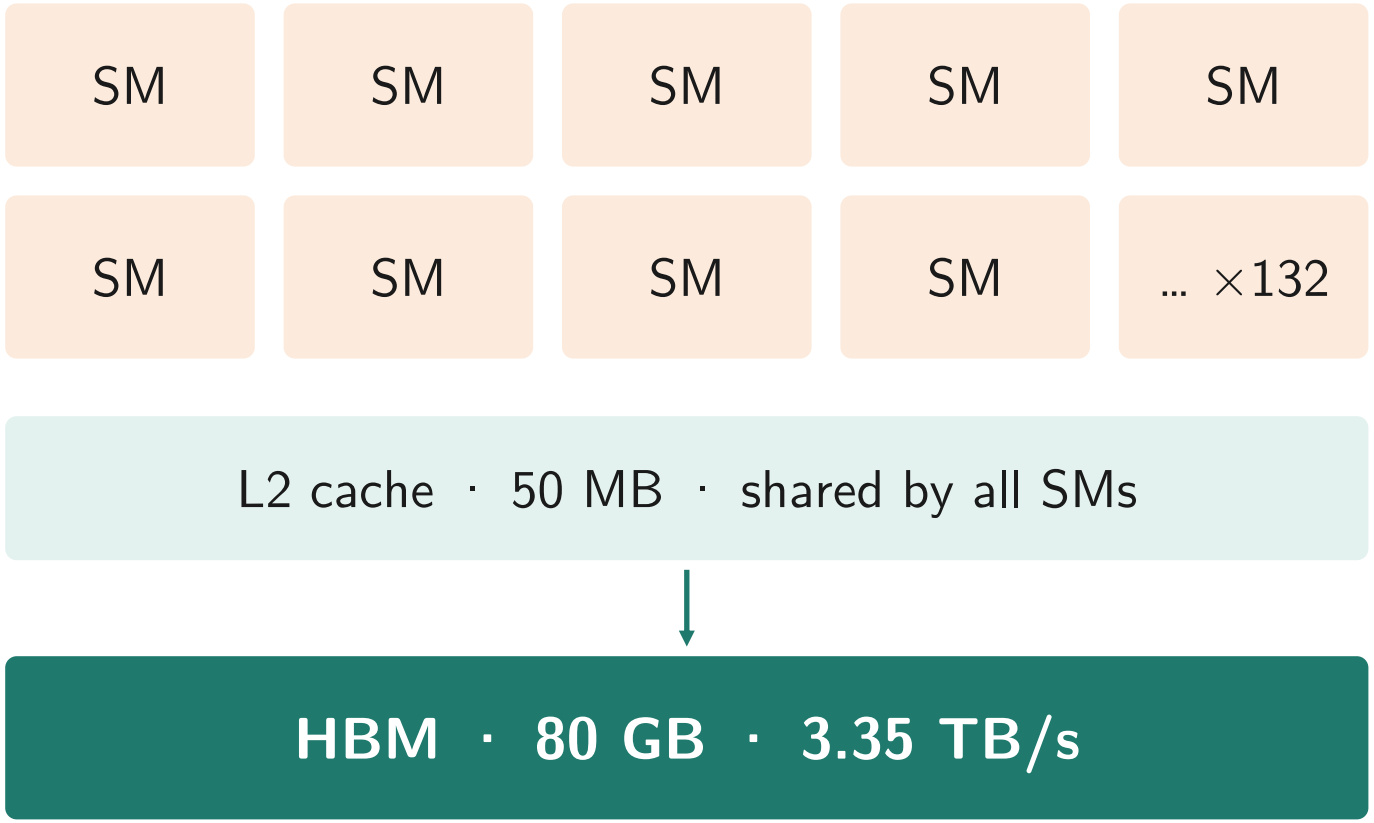

1 · From CPU to GPU

**2** · Inside the GPU

**3** · Threads, warps, blocks, and grids

**Build** · Map a grid, cost a branch

# A GPU is a grid of streaming multiprocessors




$$\begin{aligned}
&\text{Peak FP32} \\
&= \text{SMs} \times \text{lanes} \times 2 \times \text{clock} \\
&= 132 \times 128 \times 2 \times 1.98\ \text{GHz} \\
&\approx 67\ \text{TFLOP/s}
\end{aligned}$$

A fused multiply-add $a \cdot b + c$ counts as 2 FLOPs.

Source: NVIDIA H100 Tensor Core GPU datasheet (SXM5); NVIDIA H100 Tensor Core GPU Architecture whitepaper (2022).

# Inside one SM

One of 4 sub-partitions

| | |
|---|---|
| Warp scheduler | → Picks a ready warp and issues **one instruction** each cycle. |
| Registers · 64 KB | → Private, fastest storage: each thread has its own. |
| 32 CUDA cores (FP32) | → Execute that instruction for **32 threads at once**. |
| **Tensor core** | → Multiplies small matrix tiles in one step. |

× 4 sub-partitions share 256 KB of L1 / shared memory (up to 228 KB shared)

Source: NVIDIA H100 Tensor Core GPU Architecture whitepaper (2022), SM description.

# CUDA cores and tensor cores

**CUDA cores**

128 per SM · 16,896 in all

one multiply-add per lane per cycle

**any** arithmetic: adds, tanh, compares

FP32 peak ≈ **67 TFLOP/s**

**Tensor cores**

4 per SM · 528 in all

one small matrix product per instruction

**only** matrix products

BF16 peak ≈ **989 TFLOP/s**

$$989 \;/\; 67 \approx 15\times$$

**Key idea**

Matrix products run ≈ **15×** faster than other arithmetic. Number formats and tile shapes: Week 14.

Source: H100 datasheet (SXM5), dense peaks (sparse ÷ 2); H100 whitepaper (4 tensor cores per SM).

# Check yourself

Two minutes with a neighbour.

- Which work can run on tensor cores, and which cannot?
- Where does the 67 TFLOP/s FP32 peak come from?
- 4 schedulers, each issuing one 32-lane multiply-add per cycle: how many FLOPs per cycle per SM?

Part 3

# Threads, warps, blocks, and grids

How does one program become 100,000 threads?


1 · From CPU to GPU

2 · Inside the GPU

3 · Threads, warps, blocks, and grids

**Build** · Map a grid, cost a branch

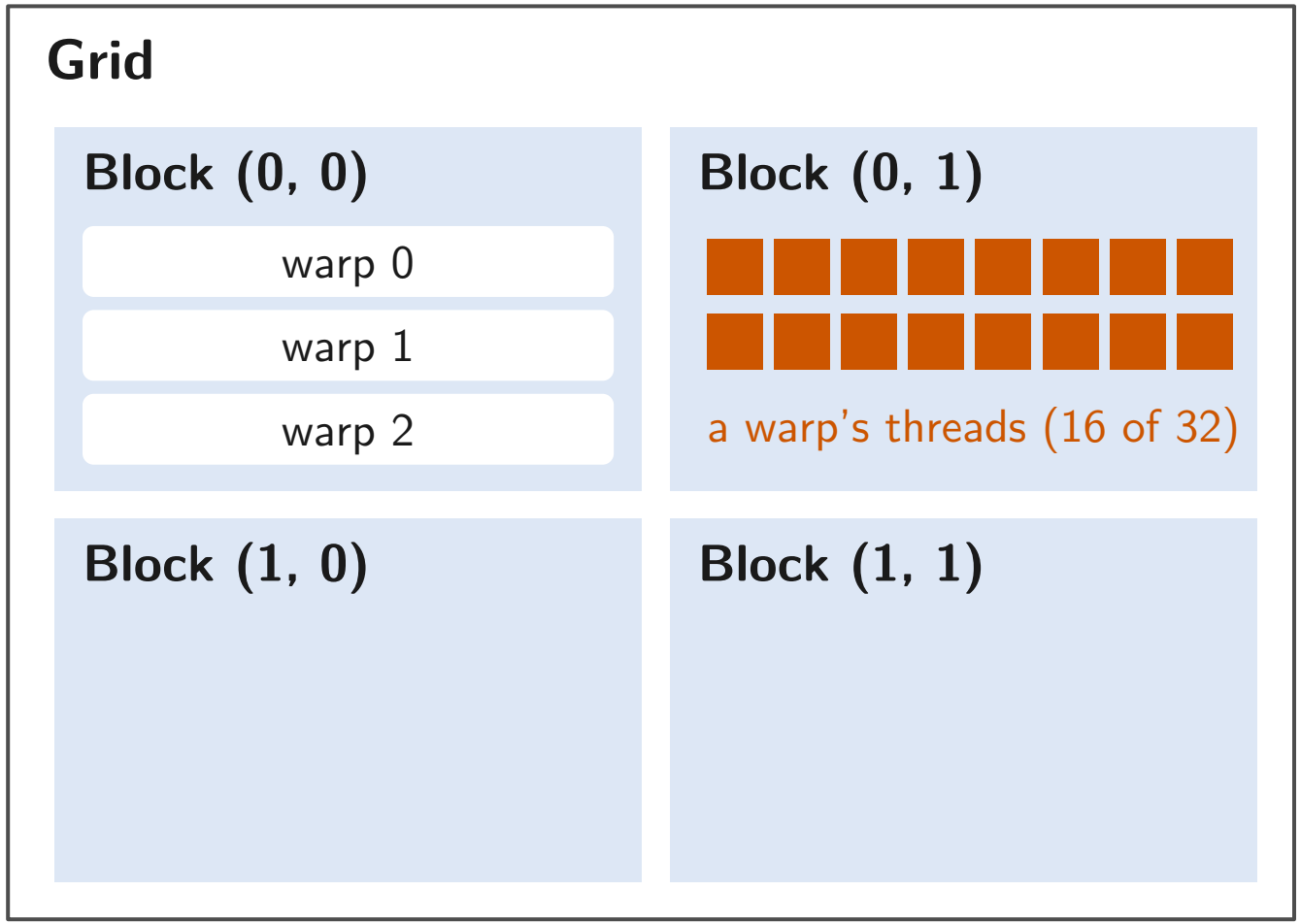
# A GPU thread is not a Week 5 thread

| | OS thread (Week 5) | GPU thread |
|---|---|---|
| How many | a handful per program | **millions** per launch |
| Created by | the operating system | the hardware, at launch |
| Switching | OS scheduler, microseconds | warp scheduler, **free** |
| Runs | on its own | in lockstep with **31 others** |

**Key idea**

A GPU thread is **cheap**: one instance of the kernel with its own registers and index.

# Software view: grid → block → warp → thread




**Grid** — every block of one launch; blocks must be **independent**.

**Block** — up to 1024 threads on **one SM**; can cooperate.

**Warp** — 32 threads that the hardware runs **together**.

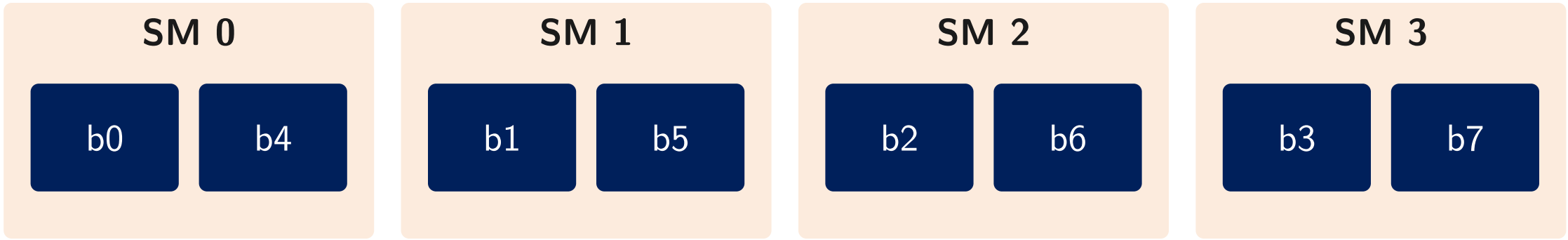
**Thread** — one instance of the kernel, with its own registers.

Source: NVIDIA CUDA C++ Programming Guide: warp size 32; at most 1024 threads per block.

# Blocks are dealt out to SMs

b0 b1 b2 b3 b4 b5 b6 b7 b8 b9 b10 b11

the grid: 12 blocks




waiting: b8 b9 b10 b11 each starts when a slot frees up

**Key idea**

A block runs on **one** SM, start to finish, in **any** order — so blocks must be independent.

Source: NVIDIA CUDA C++ Programming Guide, technical specifications for compute capability 9.0 (H100).

# Big grids run in waves

$$\text{per SM:}\quad 2{,}048 \;/\; 256 = 8 \text{ blocks}$$

$$\text{one wave:}\quad 132 \times 8 = 1{,}056 \text{ blocks}$$

$$n = 10^8\text{:}\quad 390{,}625 \text{ blocks} \approx 370 \text{ waves}$$

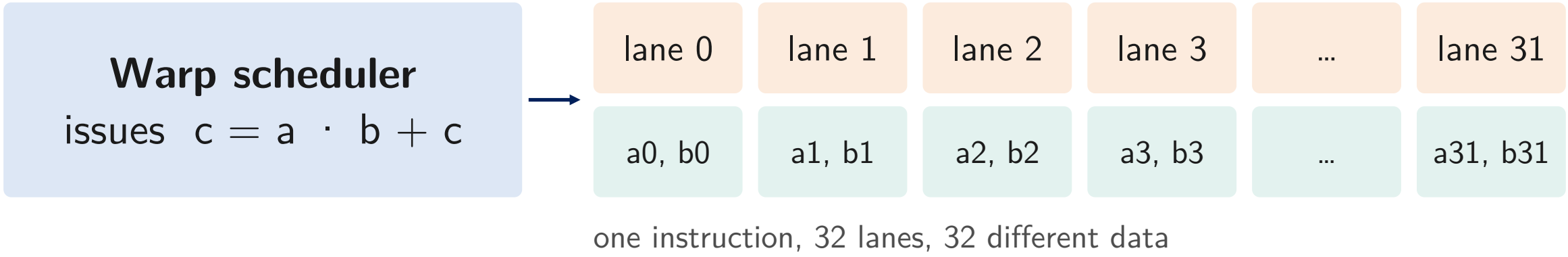
**1,056 blocks** $\rightarrow$ 1 wave
**1,057 blocks** $\rightarrow$ 2 waves, the 2nd with one block

**Watch out**

Many waves: the last partial wave hardly matters. **One or two** waves: it can double the time.

Source: NVIDIA CUDA C++ Programming Guide, technical specifications for compute capability 9.0 (H100). Assumes threads are the only limit.

# SIMT: single instruction, multiple threads




- One fetch and decode drives **32** operations $\rightarrow$ cheap control.
- Each thread has **its own registers** and index; code is written for one thread.
- A warp computes $(f(x_0), f(x_1), \ldots, f(x_{31}))$ with one instruction.

# Branch divergence

```
i = thread_index()
if x[i] > 0:          # lanes 0-15
    y[i] = slow_a(x[i])
else:                 # lanes 16-31
    y[i] = slow_b(x[i])
```

$$\text{efficiency} = \text{active lanes} \;/\; 32$$

$$50/50 \text{ split:}$$

$$\text{time} \approx \mathcal{T}(A) + \mathcal{T}(B)$$

**Pass 1: lanes run branch A**



**Pass 2: lanes run branch B**



orange: active lane · grey: masked off

**Watch out**

Divergence costs time only **within a warp**; different warps may take different paths freely.

# Whether a warp diverges depends on the data

| Condition, and the data | Warps that diverge | Time per warp |
|---|---|---|
| constant on runs of 32 | none | T(A) or T(B) |
| `x[i] > 0`, random signs | almost all | T(A) + T(B) |
| `x[i] > 0`, x sorted | at most one | T(A) or T(B) |

$$\text{random signs:}\quad \mathcal{P}(\text{all 32 agree}) = 2 \cdot 0.5^{32} \approx 4.7 \times 10^{-10}$$

**Key idea**

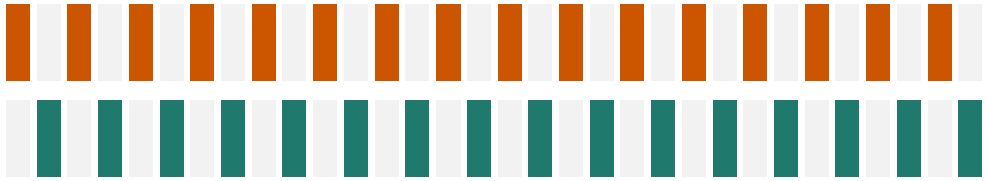
Arrange the data so that **neighbouring threads agree**; sorting or partitioning removes divergence.

# Back to the puzzle

**Kernel 1:** `i % 2 == 0`

warp 0, pass A

warp 0, pass B



**Kernel 2:** `(i // 32) % 2 == 0`

warp 0, A only

warp 1, B only



$$\text{kernel 1:}\quad \mathcal{T}(A) + \mathcal{T}(B)$$

$$\text{kernel 2:}\quad (\mathcal{T}(A) + \mathcal{T}(B)) \,/\, 2$$

$$\text{per warp} \;\Rightarrow\; 2\times \text{ faster}$$

**Key idea**

Same work, same branches: what matters is whether the **32 threads of one warp** agree.

# Check yourself

Two minutes with a neighbour.

- n $= 5000$, 256 threads per block: how many blocks, warps, and idle threads?
- Can block 7 safely read a value that block 3 writes in the same launch?
- `if x[i] > 0` on a **sorted** `x`: how many warps diverge?

IN-CLASS BUILD

# Map a grid, cost a branch

Work in pairs, on paper, about eight minutes.


1 · From CPU to GPU

2 · Inside the GPU

3 · Threads, warps, blocks, and grids

**Build** · Map a grid, cost a branch

# Exercise: map a grid, cost a branch

- n $= 10^6$, 256 threads per block: how many **blocks**, **warps**, and **idle threads**?
- How many blocks run at once on 132 SMs? How many **waves**?
- `if i % 4 == 0: A() else: B()`, A $= 100$ cycles, B $= 20$ cycles. Cycles per warp? Fraction of lane-cycles that do useful work?
- Rewrite the condition so that no warp diverges. What is the speedup?

**USE**

132 SMs $\cdot$ $\leq$ 2,048 threads per SM $\cdot$ warp $= 32$ threads

# Exercise: solutions

| Question | Answer |
|---|---|
| 1. Blocks, warps, idle | 3,907 blocks · 31,256 warps · 192 idle threads |
| 2. At once, waves | $8 \times 132 = 1{,}056$ blocks · 4 waves (last: 739) |
| 3. Divergent branch | $100 + 20 = 120$ cycles · 1,280 / 3,840 = 33% useful |
| 4. `(i // 32) % 4 == 0` | $\frac{1}{4} \cdot 100 + \frac{3}{4} \cdot 20 = 40$ cycles per warp $\rightarrow$ **3×** |

**Key idea**

Count with ceilings; ask what the **32 threads of one warp** do. Lecture 2 adds what the warps **wait** for.

# Key takeaways

- GPUs trade single-thread **latency** for **throughput**; training is a throughput problem.
- Amdahl (Week 5): the CPU side of a step bounds what any GPU can do.
- An H100 is **132 SMs**: **CUDA cores** for any arithmetic, **tensor cores** for matrix products.
- **Grid** $\rightarrow$ **block** $\rightarrow$ **warp** $\rightarrow$ **thread**: a block lives on one SM; big grids run in **waves**.
- **SIMT**: one instruction per warp of 32; **divergence** within a warp runs both paths.

# Next time, and readings

| When | What |
|---|---|
| Lecture 2 (this week) | Execution model: latency hiding, occupancy, launches, copies |
| Friday (this week) | T3 due |
| After Lecture 2 | T1 and Lab 3 released |
| Week 13 | The memory hierarchy; arithmetic intensity and the roofline |

**READ BEFORE LECTURE 2**

CUDA C++ Programming Guide: *Hardware Implementation* (SIMT, warps).

PyTorch documentation: *CUDA semantics* (asynchronous execution).